

Classes and Objects: A Deeper Look

Chapter 10 of Visual C# How to Program, 6/e

OBJECTIVES

In this chapter you'll:

- Use composition to allow a class to have references to objects of other classes as members.
- Throw an exception to indicate that an argument is out of range.
- Enable an object to refer to itself with the keyword `this`.
- Use `static` variables and methods.
- Use `readonly` fields.
- Take advantage of C#'s memory-management features.
- Use the IDE's **Class View** and **Object Browser** windows.
- Use object initializers to create an object and initialize it in the same statement.
- Overload built-in operators to work with objects of your own types.
- Define your own value type with `struct`.
- Use extension methods to enhance an existing class's capabilities.

10.1 Introduction

10.2 Time Class Case Study; Throwing Exceptions

10.2.1 `Time1` Class Declaration

10.2.2 Using Class `Time1`

10.3 Controlling Access to Members

10.4 Referring to the Current Object's Members with the `this` Reference

10.5 Time Class Case Study: Overloaded Constructors

10.5.1 Class `Time2` with Overloaded Constructors

10.5.2 Using Class `Time2`'s Overloaded Constructors

10.6 Default and Parameterless Constructors

10.7 Composition

10.7.1 Class `Date`

10.7.2 Class `Employee`

10.7.3 Class `EmployeeTest`

10.8 Garbage Collection and Destructors

10.8 Garbage Collection and Destructors

10.9 `static` Class Members

10.10 `readonly` Instance Variables

10.11 **Class View** and **Object Browser**

10.11.1 Using the **Class View** Window

10.11.2 Using the **Object Browser**

10.12 Object Initializers

10.13 Operator Overloading; Introducing `struct`

10.13.1 Creating Value Types with `struct`

10.13.2 Value Type `ComplexNumber`

10.13.3 Class `ComplexTest`

10.14 **Time** Class Case Study: Extension Methods

10.15 Wrap-Up

10.2 Time Class Case Study; Throwing Exceptions

- ▶ Class `Time1` (Fig. 10.1) represents the time of day.
- ▶ The class contains `public` properties for the `Hour`, `Minute` and `Second`.
- ▶ A class's `public` members are the `public` services or the `public` interface that this class provides to its clients.
- ▶ `Time1` does not declare a constructor, so the compiler defines a default constructor.
- ▶ Each property receives the default value `0` for an `int`.
- ▶ Instance variables and auto-implemented properties also can be assigned values in their declarations

```
1  // Fig. 10.1: Time1.cs
2  // Time1 class declaration maintains the time in 24-hour format.
3  using System; // namespace containing ArgumentOutOfRangeException
4
5  public class Time1
6  {
7      public int Hour { get; set; } // 0 - 23
8      public int Minute { get; set; } // 0 - 59
9      public int Second { get; set; } // 0 - 59
10
```

Fig. 10.1 | Time1 class declaration maintains the time in 24-hour format. (Part 1 of 3.)

```
11 // set a new time value using universal time; throw an
12 // exception if the hour, minute or second is invalid
13 public void SetTime(int hour, int minute, int second)
14 {
15     // validate hour, minute and second
16     if ((hour < 0 || hour > 23) || (minute < 0 || minute > 59) ||
17         (second < 0 || second > 59))
18     {
19         throw new ArgumentOutOfRangeException();
20     }
21
22     Hour = hour;
23     Minute = minute;
24     Second = second;
25 }
26
```

Fig. 10.1 | Time1 class declaration maintains the time in 24-hour format. (Part 2 of 3.)

```
27 // convert to string in universal-time format (HH:MM:SS)
28 public string ToUniversalString() =>
29     $"{Hour:D2}:{Minute:D2}:{Second:D2}";
30
31 // convert to string in standard-time format (H:MM:SS AM or PM)
32 public override string ToString() =>
33     $"{((Hour == 0 || Hour == 12) ? 12 : Hour % 12)}:" +
34     $"{Minute:D2}:{Second:D2} {(Hour < 12 ? "AM" : "PM")}";
35 }
```

Fig. 10.1 | Time1 class declaration maintains the time in 24-hour format. (Part 3 of 3.)

10.2 Time Class Case Study (Cont.)

public Class

- ▶ Time1 is a **public** class—it can potentially can be reused in other projects.
- ▶ Although we use class Time1 only in this project, from this point forward, we'll declare as `public` any class that could potentially be reused in another project.

10.2 Time Class Case Study (Cont.)

Method setTime and Throwing Exceptions

- ▶ Method `setTime` is a `public` method that declares three `int` parameters and uses them to set the time.
- ▶ Lines 16–17 test each argument to determine whether the value is out of range. If not, lines 22–24 assign values to the properties.
- ▶ The hour value must be greater than or equal to 0 and less than 24, because universal-time format represents hours as integers from 0 to 23 (e.g., 1 PM is hour 13 and 11 PM is hour 23; midnight is hour 0 and noon is hour 12).

10.2 Time Class Case Study (Cont.)

- ▶ Both minute and second values must be greater than or equal to 0 and less than 60.
- ▶ For values outside these ranges, `SetTime` throws an `ArgumentOutOfRangeException`, which notifies the client code that an invalid argument was passed to the method.
- ▶ The `throw` statement creates a new object of type `ArgumentOutOfRangeException`.
- ▶ The parentheses following the class name indicate a call to the `ArgumentOutOfRangeException` constructor.

10.2 Time Class Case Study (Cont.)

- ▶ After the exception object is created, the `throw` statement immediately terminates method `SetTime` and the exception is returned to the code that attempted to set the time.

10.2 Time Class Case Study (Cont.)

Method ToUniversalString

- ▶ Method ToUniversalString is an expression-bodied method
- ▶ The method takes no arguments and returns a `string` in universal-time format, consisting of six digits—two for the hour, two for the minute and two for the second.
- ▶ The D2 format specifier formats an integer with two digits and, where needed, a leading 0 if the integer has fewer than two digits.

10.2 Time Class Case Study (Cont.)

Method ToString

- ▶ Method ToString (lines 32–34) is an expression-bodied method that takes no arguments and returns a `string` in which the `Hour`, `Minute` and `Second` values are separated by colons and followed by an `AM` or `PM` indicator (e.g., `1:27:06 PM`).

10.2 Time Class Case Study (Cont.)

- ▶ The `Time1Test` app class (Fig. 10.2) uses class `Time1`.

```
1  // Fig. 10.2: Time1Test.cs
2  // Time1 object used in an app.
3  using System;
4
5  class Time1Test
6  {
7      static void Main()
8      {
9          // create and initialize a Time1 object
10         var time = new Time1(); // invokes Time1 constructor
11
12         // output string representations of the time
13         Console.WriteLine(
14             $"The initial universal time is: {time.ToUniversalString()}");
15         Console.WriteLine(
16             $"The initial standard time is: {time.ToString()}");
17         Console.WriteLine(); // output a blank line
18     }
```

Fig. 10.2 | Time1 object used in an app. (Part 1 of 3.)

```
19 // change time and output updated time
20 time.SetTime(13, 27, 6);
21 Console.WriteLine(
22     $"Universal time after SetTime is: {time.ToUniversalString()}");
23 Console.WriteLine(
24     $"Standard time after SetTime is: {time.ToString()}");
25 Console.WriteLine(); // output a blank line
26
27 // attempt to set time with invalid values
28 try
29 {
30     time.SetTime(99, 99, 99);
31 }
32 catch (ArgumentOutOfRangeException ex)
33 {
34     Console.WriteLine(ex.Message + "\n");
35 }
```

Fig. 10.2 | Time1 object used in an app. (Part 2 of 3.)

```
36
37     // display time after attempt to set invalid values
38     Console.WriteLine("After attempting invalid settings:");
39     Console.WriteLine($"Universal time: {time.ToUniversalString()}");
40     Console.WriteLine($"Standard time: {time.ToString()}");
41 }
42 }
```

The initial universal time is: 00:00:00
The initial standard time is: 12:00:00 AM

Universal time after SetTime is: 13:27:06
Standard time after SetTime is: 1:27:06 PM

Specified argument was out of the range of valid values.

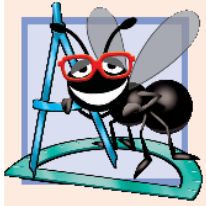
After attempting invalid settings:
Universal time: 13:27:06
Standard time: 1:27:06 PM

Fig. 10.2 | Time1 object used in an app. (Part 3 of 3.)

10.2 Time Class Case Study (Cont.)

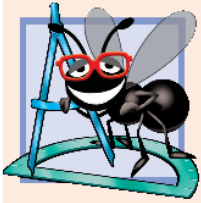
Notes on the Time1 Class Declaration

- ▶ Consider several class-design issues with respect to class `Time1`.
 - The time is represented as three integers for the hour, minute and second.
 - The actual data representation used within the class is of no concern to the class's clients.
 - It would be perfectly reasonable for `Time1` to represent the time internally as the number of seconds since midnight or the number of minutes and seconds since midnight.
 - Clients could use the same public methods and properties to get the same results without being aware of this
 - The `Hour`, `Minute` and `Second` properties would need to be reimplemented to work with the new data representation.



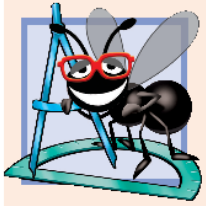
Software Engineering Observation 10.1

Classes simplify programming because the client can use only the `public` members exposed by the class. Such members are usually client oriented rather than implementation oriented. Clients are neither aware of, nor involved in, a class's implementation. Clients generally care about what the class does but not how the class does it. Clients do, of course, care that the class operates correctly and efficiently.



Software Engineering Observation 10.2

Interfaces change less frequently than implementations. When an implementation changes, implementation-dependent code must change accordingly. Hiding the implementation reduces the possibility that other parts of the app become dependent on class-implementation details.



Software Engineering Observation 10.3

Date and time manipulations are more complex than the simplified classes we use in this book. For applications that require date and time processing, check out .NET's `DateTimeOffset`, `DateTime`, `TimeSpan` and `TimeZoneInfo` value types in namespace `System`.

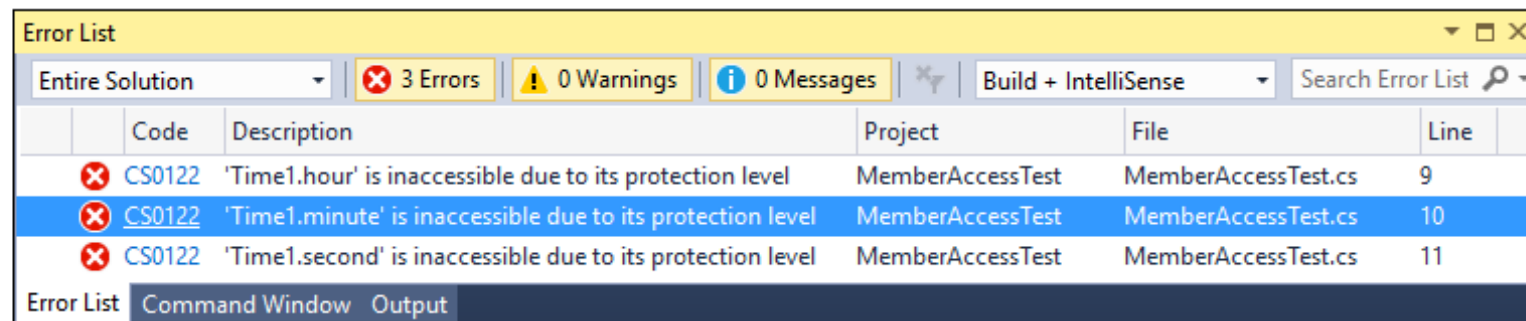
10.3 Controlling Access to Members

- ▶ The access modifiers `public` and `private` control access to a class's variables, methods and properties.
- ▶ The primary purpose of `public` methods/properties is to present to the class's clients a view of the services the class provides.
 - Clients of the class need not be concerned with how the class accomplishes its tasks.
- ▶ A class's `private` variables, properties and methods are not directly accessible to the class's clients.

10.3 Controlling Access to Members

- ▶ Figure 10.3 demonstrates that `private` class members are not directly accessible outside the class.
- ▶ In this app, we use a modified version of class `Time1` that declares `private` instance variables `hour`, `minute` and `second`, rather than public properties `Hour`, `Minute` and `Second`.
- ▶ Members of a class—for instance, properties, methods and instance variables—have private access by default.


```
1 // Fig. 10.3: MemberAccessTest.cs
2 // Private members of class Time1 are not accessible outside the class.
3 class MemberAccessTest
4 {
5     static void Main()
6     {
7         var time = new Time1(); // create and initialize Time1 object
8
9         time.hour = 7; // error: hour has private access in Time1
10        time.minute = 15; // error: minute has private access in Time1
11        time.second = 30; // error: second has private access in Time1
12    }
13 }
```



The screenshot shows the 'Error List' window in Visual Studio. It displays three errors, all of type CS0122, indicating that 'Time1.hour', 'Time1.minute', and 'Time1.second' are inaccessible due to their protection level. The errors are listed in a table with columns for Code, Description, Project, File, and Line.

Code	Description	Project	File	Line
CS0122	'Time1.hour' is inaccessible due to its protection level	MemberAccessTest	MemberAccessTest.cs	9
CS0122	'Time1.minute' is inaccessible due to its protection level	MemberAccessTest	MemberAccessTest.cs	10
CS0122	'Time1.second' is inaccessible due to its protection level	MemberAccessTest	MemberAccessTest.cs	11

Fig. 10.3 | Private members of class `Time1` are not accessible outside the class.

10.4 Referring to the Current Object's Members with the `this` Reference

- ▶ Every object can access a reference to itself with keyword `this`.
- ▶ When a non-static method/property is called, its body implicitly uses keyword `this` to refer to the object's instance variables and other non-static class members.
- ▶ As you'll see in Fig. 10.4, you can also use keyword `this` explicitly in a non-static method's body.

```
1  // Fig. 10.4: ThisTest.cs
2  // this used implicitly and explicitly to refer to members of an object.
3  using System;
4
5  class ThisTest
6  {
7      static void Main()
8      {
9          var time = new SimpleTime(15, 30, 19);
10         Console.WriteLine(time.BuildString());
11     }
12 }
13
```

Fig. 10.4 | this used implicitly and explicitly to refer to members of an object. (Part 1 of 3.)

```
14 // class SimpleTime demonstrates the "this" reference
15 public class SimpleTime
16 {
17     private int hour; // 0-23
18     private int minute; // 0-59
19     private int second; // 0-59
20
21     // if the constructor uses parameter names identical to
22     // instance-variable names, the "this" reference is
23     // required to distinguish between the names
24     public SimpleTime(int hour, int minute, int second)
25     {
26         this.hour = hour; // set "this" object's hour instance variable
27         this.minute = minute; // set "this" object's minute
28         this.second = second; // set "this" object's second
29     }
30
```

Fig. 10.4 | this used implicitly and explicitly to refer to members of an object. (Part 2 of 3.)

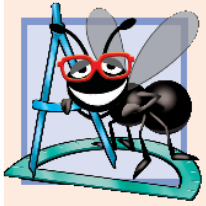
```
31 // use explicit and implicit "this" to call ToUniversalString
32 public string BuildString() =>
33     $"{"this.ToUniversalString()",24}: {this.ToUniversalString()}" +
34     $"\\n{"ToUniversalString()",24}: {ToUniversalString()}";
35
36 // convert to string in universal-time format (HH:MM:SS);
37 // "this" is not required here to access instance variables,
38 // because the method does not have local variables with the same
39 // names as the instance variables
40 public string ToUniversalString() =>
41     $"{"this.hour:D2}:{this.minute:D2}:{this.second:D2}";
42 }
```

```
this.ToUniversalString(): 15:30:19
ToUniversalString(): 15:30:19
```

Fig. 10.4 | this used implicitly and explicitly to refer to members of an object. (Part 3 of 3.)

10.4 Referring to the Current Object's Members with the `this` Reference (Cont.)

- ▶ If the constructor's parameter names are identical to the class's instance-variable names, so they hide (shadow) the corresponding instance variables.
- ▶ You can use the `this` reference to refer to hidden instance variables explicitly.
- ▶ If a member is not hidden, the `this` keyword is implied, but can be included explicitly.



Software Engineering Observation 10.4

Using properties throughout a class to access the class's instance variables normally eliminates shadowing because property names use Pascal Case naming (capital first letter) and parameter names use Camel Case (lowercase first letter).

10.5 Time Class Case Study: Overloaded Constructors

- ▶ Overloaded constructors enable objects of a class to be conveniently initialized in different ways.
- ▶ To overload constructors, simply provide multiple constructor declarations with different signatures.

10.5.1 Class Time2 with Overloaded Constructors

- ▶ Class Time2 (Fig. 10.5) contains overloaded constructors.

```
1  // Fig. 10.5: Time2.cs
2  // Time2 class declaration with overloaded constructors.
3  using System; // for class ArgumentOutOfRangeException
4
5  public class Time2
6  {
7      private int hour; // 0 - 23
8      private int minute; // 0 - 59
9      private int second; // 0 - 59
10
11     // constructor can be called with zero, one, two or three arguments
12     public Time2(int hour = 0, int minute = 0, int second = 0)
13     {
14         SetTime(hour, minute, second); // invoke SetTime to validate time
15     }
16
```

Fig. 10.5 | Time2 class declaration with overloaded constructors. (Part 1 of 6.)

```
17 // Time2 constructor: another Time2 object supplied as an argument
18 public Time2(Time2 time)
19     : this(time.Hour, time.Minute, time.Second) { }
20
21 // set a new time value using universal time; invalid values
22 // cause the properties' set accessors to throw exceptions
23 public void SetTime(int hour, int minute, int second)
24 {
25     Hour = hour; // set the Hour property
26     Minute = minute; // set the Minute property
27     Second = second; // set the Second property
28 }
29
```

Fig. 10.5 | Time2 class declaration with overloaded constructors. (Part 2 of 6.)

```
30    // property that gets and sets the hour
31    public int Hour
32    {
33        get
34        {
35            return hour;
36        }
37        set
38        {
39            if (value < 0 || value > 23)
40            {
41                throw new ArgumentOutOfRangeException(nameof(value),
42                    value, $"{nameof(Hour)} must be 0-23");
43            }
44
45            hour = value;
46        }
47    }
48
```

Fig. 10.5 | Time2 class declaration with overloaded constructors. (Part 3 of 6.)

```
49 // property that gets and sets the minute
50 public int Minute
51 {
52     get
53     {
54         return minute;
55     }
56     set
57     {
58         if (value < 0 || value > 59)
59         {
60             throw new ArgumentOutOfRangeException(nameof(value),
61                 value, $"{nameof(Minute)} must be 0-59");
62         }
63
64         minute = value;
65     }
66 }
67
```

Fig. 10.5 | Time2 class declaration with overloaded constructors. (Part 4 of 6.)

```
68      // property that gets and sets the second
69      public int Second
70      {
71          get
72          {
73              return second;
74          }
75          set
76          {
77              if (value < 0 || value > 59)
78              {
79                  throw new ArgumentOutOfRangeException(nameof(value),
80                  value, $"{nameof(Second)} must be 0-59");
81              }
82
83              second = value;
84          }
85      }
86
```

Fig. 10.5 | Time2 class declaration with overloaded constructors. (Part 5 of 6.)

```
87 // convert to string in universal-time format (HH:MM:SS)
88 public string ToUniversalString() =>
89     $"{Hour:D2}:{Minute:D2}:{Second:D2}";
90
91 // convert to string in standard-time format (H:MM:SS AM or PM)
92 public override string ToString() =>
93     $"{((Hour == 0 || Hour == 12) ? 12 : Hour % 12)}:" +
94     $"{Minute:D2}:{Second:D2} {(Hour < 12 ? "AM" : "PM")}";
95 }
```

Fig. 10.5 | Time2 class declaration with overloaded constructors. (Part 6 of 6.)

10.5 Time Class Case Study: Overloaded Constructors (Cont.)

Class Time2's Three-Argument Constructor

- ▶ Lines 12–15 declare a constructor with three default parameters.
- ▶ This is also the class's parameterless constructor because you can call the constructor without arguments and the compiler will automatically provide the default parameter values.

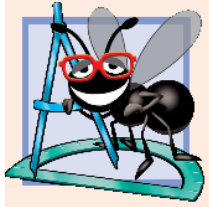


Common Programming Error 10.1

A constructor can call methods of its class. Be aware that the instance variables might not yet be initialized, because the constructor is in the process of initializing the object. Using instance variables before they have been initialized properly is a logic error.

10.5 Time Class Case Study: Overloaded Constructors (Cont.)

- ▶ Using this as in line 19
 - `: this(time.Hour, time.Minute, time.Second) { }`
- ▶ is called a **constructor initializer**.
- ▶ Enables a class to reuse initialization code provided by a constructor, rather than defining similar code in another constructor.



Software Engineering Observation 10.5

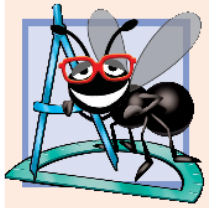
Constructor initializers make classes easier to maintain, modify and debug, because the common initialization code can be defined in one constructor and called by others.

10.5 Time Class Case Study: Overloaded Constructors (Cont.)

- ▶ Notes Regarding Class Time2's Methods, Properties and Constructors
- ▶ Consider changing the representation of the time to a single int value representing the total number of seconds that have elapsed since midnight.
 - Only the bodies of the methods that access the private data directly would need to change.
 - There would be no need to modify the bodies of methods SetTime, ToUniversalString or ToString.
- ▶ When there is no access modifier before a get or set accessor, the accessor inherits the access modifier preceding the property name.

10.5 Time Class Case Study: Overloaded Constructors (Cont.)

- ▶ It's common to include in an exception's error message a variable's or property's identifier.
 - Can help a client-code programmer understand the context in which the exception occurred.
- ▶ Prior to C# 6, you had to hard code these identifiers into your error-messages.
- ▶ As of C# 6, you can instead use the `nameof` operator, which returns a `string` representation of the identifier enclosed in parentheses.



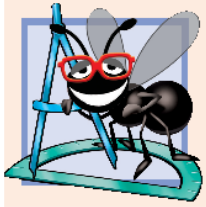
Software Engineering Observation 10.6

*When executing a method of a class, if that method has a reference to another object of the same class (typically received via a parameter), the method can access all of that other object's data and methods (including those that are **private**).*



Good Programming Practice 10.1

When you need to include an identifier in a `string` literal, use the `nameof` operator rather than hard coding the identifier's name in the `string`. If you right click an identifier in Visual Studio then use the `Rename...` option to change the identifier's name throughout your code, the `string` that `nameof` returns will be updated automatically with the identifier's new name.



Software Engineering Observation 10.7

*When implementing a method of a class, using the class's properties to access the class's **private** data simplifies code maintenance and reduces the likelihood of errors.*

10.6 Time Class Case Study: Overloaded Constructors (Cont.)

- ▶ Class Time2Test (Fig. 10.6) creates six Time2 objects to invoke the overloaded Time2 constructors.

```
1  // Fig. 10.6: Time2Test.cs
2  // Overloaded constructors used to initialize Time2 objects.
3  using System;
4
5  public class Time2Test
6  {
7      static void Main()
8      {
9          var t1 = new Time2(); // 00:00:00
10         var t2 = new Time2(2); // 02:00:00
11         var t3 = new Time2(21, 34); // 21:34:00
12         var t4 = new Time2(12, 25, 42); // 12:25:42
13         var t5 = new Time2(t4); // 12:25:42
14     }
```

Fig. 10.6 | Overloaded constructors used to initialize Time2 objects. (Part 1 of 4.)

```
15 Console.WriteLine("Constructed with:\n");
16 Console.WriteLine("t1: all arguments defaulted");
17 Console.WriteLine($"    {t1.ToUniversalString()}"); // 00:00:00
18 Console.WriteLine($"    {t1.ToString()}\n"); // 12:00:00 AM
19
20 Console.WriteLine(
21     "t2: hour specified; minute and second defaulted");
22 Console.WriteLine($"    {t2.ToUniversalString()}"); // 02:00:00
23 Console.WriteLine($"    {t2.ToString()}\n"); // 2:00:00 AM
24
25 Console.WriteLine(
26     "t3: hour and minute specified; second defaulted");
27 Console.WriteLine($"    {t3.ToUniversalString()}"); // 21:34:00
28 Console.WriteLine($"    {t3.ToString()}\n"); // 9:34:00 PM
29
```

Fig. 10.6 | Overloaded constructors used to initialize Time2 objects. (Part 2 of 4.)

```
30 Console.WriteLine("t4: hour, minute and second specified");
31 Console.WriteLine($" {t4.ToUniversalString()}"); // 12:25:42
32 Console.WriteLine($" {t4.ToString()}\\n"); // 12:25:42 PM
33
34 Console.WriteLine("t5: Time2 object t4 specified");
35 Console.WriteLine($" {t5.ToUniversalString()}"); // 12:25:42
36 Console.WriteLine($" {t5.ToString()}"); // 12:25:42 PM
37
38 // attempt to initialize t6 with invalid values
39 try
40 {
41     var t6 = new Time2(27, 74, 99); // invalid values
42 }
43 catch (ArgumentOutOfRangeException ex)
44 {
45     Console.WriteLine("\\nException while initializing t6:");
46     Console.WriteLine(ex.Message);
47 }
48 }
49 }
```

Fig. 10.6 | Overloaded constructors used to initialize Time2 objects. (Part 3 of 4.)

Constructed with:

t1: all arguments defaulted

00:00:00

12:00:00 AM

t2: hour specified; minute and second defaulted

02:00:00

2:00:00 AM

t3: hour and minute specified; second defaulted

21:34:00

9:34:00 PM

t4: hour, minute and second specified

12:25:42

12:25:42 PM

t5: Time2 object t4 specified

12:25:42

12:25:42 PM

Exception while initializing t6:

Hour must be 0-23

Parameter name: value

Actual value was 27.

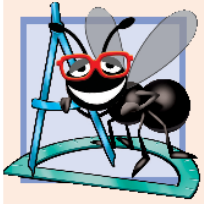
Fig. 10.6 | Overloaded constructors used to initialize Time2 objects. (Part 4 of 4.)

10.6 Default and Parameterless Constructors

- Every class *must* have at least one constructor.
- If you do not provide any constructors in a class's declaration, the compiler creates a *default constructor* that takes no arguments when it is invoked.
- The compiler will *not* create a default constructor for a class that explicitly declares at least one constructor.
- If you have declared a constructor, but want to be able to invoke the constructor with no arguments, you must declare a *parameterless constructor*.

10.7 Composition

- ▶ A class can have objects of values types or references to objects of other classes as members.
- ▶ This is called composition and is sometimes referred to as a *has-a* relationship.



Software Engineering Observation 10.8

*One form of software reuse is composition, in which a class contains references to other objects. Recall that classes are reference types. A class can have a property of its own type—for example, a **Person** class could have **Mother** and **Father** properties of type **Person** that reference other **Person** objects.*

10.7.1 Class Date

- ▶ Class Date (Fig. 10.7) declares instance variables `month` and `day`, and auto-implemented property `Year` (line 9) to represent a date.

```
1  // Fig. 10.7: Date.cs
2  // Date class declaration.
3  using System;
4
5  public class Date
6  {
7      private int month; // 1-12
8      private int day; // 1-31 based on month
9      public int Year { get; private set; } // auto-implemented property Year
10
11     // constructor: use property Month to confirm proper value for month;
12     // use property Day to confirm proper value for day
13     public Date(int month, int day, int year)
14     {
15         Month = month; // validates month
16         Year = year; // could validate year
17         Day = day; // validates day
18         Console.WriteLine($"Date object constructor for date {this}");
19     }
```

Fig. 10.7 | Date class declaration. (Part I of 4.)

```
20
21 // property that gets and sets the month
22 public int Month
23 {
24     get
25     {
26         return month;
27     }
28     private set // make writing inaccessible outside the class
29     {
30         if (value <= 0 || value > 12) // validate month
31         {
32             throw new ArgumentOutOfRangeException(
33                 nameof(value), value, $"{nameof(Month)} must be 1-12");
34         }
35
36         month = value;
37     }
38 }
```

Fig. 10.7 | Date class declaration. (Part 2 of 4.)

```
39
40 // property that gets and sets the day
41 public int Day
42 {
43     get
44     {
45         return day;
46     }
47     private set // make writing inaccessible outside the class
48     {
49         int[] daysPerMonth =
50             {0, 31, 29, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
51
52         // check if day in range for month
53         if (value <= 0 || value > daysPerMonth[Month])
54         {
55             throw new ArgumentOutOfRangeException(nameof(value), value,
56                 $"{nameof(Day)} out of range for current month/year");
57         }
58     }
59 }
```

Fig. 10.7 | Date class declaration. (Part 3 of 4.)

```
58         // check for leap year
59         if (Month == 2 && value == 29 &&
60             !(Year % 400 == 0 || (Year % 4 == 0 && Year % 100 != 0)))
61         {
62             throw new ArgumentOutOfRangeException(nameof(value), value,
63                 $"{nameof(Day)} out of range for current month/year");
64         }
65
66         day = value;
67     }
68 }
69
70 // return a string of the form month/day/year
71 public override string ToString() => $"{Month}/{Day}/{Year}";
72 }
```

Fig. 10.7 | Date class declaration. (Part 4 of 4.)

10.7.2 Class Employee

- ▶ Class Employee (Fig. 10.8) has `public` auto-implemented, getter-only properties `FirstName`, `LastName`, `BirthDate` and `HireDate`
- ▶ Demonstrates that *a class can have references to objects of other classes as members*

```
1  // Fig. 10.8: Employee.cs
2  // Employee class with references to other objects.
3  public class Employee
4  {
5      public string FirstName { get; }
6      public string LastName { get; }
7      public Date BirthDate { get; }
8      public Date HireDate { get; }
9
10     // constructor to initialize name, birth date and hire date
11     public Employee(string firstName, string lastName,
12         Date birthDate, Date hireDate)
13     {
14         FirstName = firstName;
15         LastName = lastName;
16         BirthDate = birthDate;
17         HireDate = hireDate;
18     }
19
20     // convert Employee to string format
21     public override string ToString() => $"{LastName}, {FirstName} " +
22         $"Hired: {HireDate}   Birthday: {BirthDate}";
23 }
```

Fig. 10.8 | Employee class with references to other objects.

10.7.3 Class EmployeeTest

- ▶ Class EmployeeTest (Fig. 10.9) creates two Date objects to represent an Employee's birthday and hire date, respectively.


```
1  // Fig. 10.9: EmployeeTest.cs
2  // Composition demonstration.
3  using System;
4
5  class EmployeeTest
6  {
7      static void Main()
8      {
9          var birthday = new Date(7, 24, 1949);
10         var hireDate = new Date(3, 12, 1988);
11         var employee = new Employee("Bob", "Blue", birthday, hireDate);
12
13         Console.WriteLine(employee);
14     }
15 }
```

Date object constructor for date 7/24/1949
Date object constructor for date 3/12/1988
Blue, Bob Hired: 3/12/1988 Birthday: 7/24/1949

Fig. 10.9 | Composition demonstration.

10.8 Garbage Collection and Destructors

- Every object you create uses various system resources, such as memory.
- In many programming languages, these system resources are reserved for the object's use until they're explicitly released by the programmer.
- If all the references to the object that manages the resource are lost before the resource is explicitly released, it can no longer be released. This is known as a **resource leak**.

10.8 Garbage Collection and Destructors

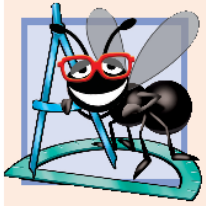
- The Common Language Runtime (CLR) uses a **garbage collector** to *reclaim* the memory occupied by objects that are no longer in use.
- When there are no more references to an object, the object becomes **eligible for destruction**.
- Every object has a **destructor** that's invoked by the garbage collector to perform **termination housekeeping** before its memory is reclaimed.

10.8 Garbage Collection and Destructors (Cont.)

- A destructor's name is the class name, preceded by a tilde, and it has no access modifier in its header.
- After an object's destructor is called, the object becomes **eligible for garbage collection**—the memory for the object can be reclaimed by the garbage collector.
- **Memory leaks** are less likely in C# than languages like C and C++ (but some can still happen in subtle ways).

10.8 Garbage Collection and Destructors (Cont.)

- Other types of resource leaks can occur, for example if an app fails to close a file that it has opened.
- A problem with the garbage collector is that it is not guaranteed to perform its tasks at a specified time. For this reason, destructors are rarely used.

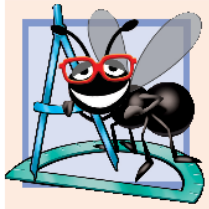


Software Engineering Observation 10.9

A class that uses resources, such as files on disk, should provide a method to eventually release the resources. Many Framework Class Library classes provide `Close` or `Dispose` methods for this purpose. Section 13.6 introduces the `Dispose` method, which is then used in many later examples. `Close` methods are typically used with objects that are associated with files (Chapter 17) and other types of so-called streams of data.

10.9 static Class Members

- A `static` variable or property is used when only one copy of a particular variable should be shared by all objects of a class.
- A `static` variable or property represents **classwide information**—all objects of the class share the *same* piece of data.
- The declaration of a `static` variable or property begins with the keyword `static`.



Software Engineering Observation 10.10

Use a static variable when all objects of a class must share the same copy of the variable.

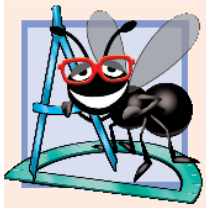
10.9 static Class Members (Cont.)

- The scope of a `static` variable is the body of its class.
- A class's `public static` members can be accessed by qualifying the member name with the class name and the member access (`.`) operator, as in `Math.PI`.
- A class's `private static` class members can be accessed only through the class's methods and properties.
- To access a `private static` member from outside its class, a `public static` method or property can be provided.



Common Programming Error 10.2

It's a compilation error to access or invoke a static member by referencing it through an instance of the class, like a non-static member.



Software Engineering Observation 10.11

static variables, methods and properties exist, and can be used, even if no objects of that class have been instantiated. static members are available as soon as the class is loaded into memory at execution time.

10.9 static Class Members (Cont.)

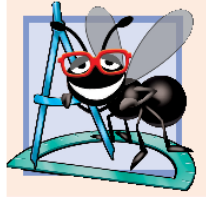
- ▶ A `static` method (or property) cannot access non-`static` class members directly, because a `static` method (or property) can be called even when *no* objects of the class exist.
- ▶ For the same reason, this cannot be used in a `static` method—the `this` reference always refers to a *specific object* of the class.
- ▶ When a `static` method is called, it does not know which object to manipulate and there might *not* be *any* objects of its class in memory.

10.9 static Class Members (cont.)

- ▶ Class `Employee` declares `private static` auto-implemented property `Count` to maintain a count of the number of `Employee` objects that have been created.
- ▶ `Count`'s `set` accessor is `private`, because only class `Employee` should be able to modify `Count`'s value.
- ▶ `Count` is a `static` auto-implemented property, so the compiler creates a corresponding `private static` variable that `Count` manages.
- ▶ When you declare a `static` variable and do not initialize it, the compiler initializes it to the type's default value (in this case, 0).

```
1  // Fig. 10.10: Employee.cs
2  // static property used to maintain a count of the number of
3  // Employee objects that have been created.
4  using System;
5
6  public class Employee
7  {
8      public static int Count { get; private set; } // objects in memory
9
10     public string FirstName { get; }
11     public string LastName { get; }
12
13     // initialize employee, add 1 to static Count and
14     // output string indicating that constructor was called
15     public Employee(string firstName, string lastName)
16     {
17         FirstName = firstName;
18         LastName = lastName;
19         ++Count; // increment static count of employees
20         Console.WriteLine("Employee constructor: " +
21             $"{FirstName} {LastName}; Count = {Count}");
22     }
23 }
```

Fig. 10.10 | static property used to maintain a count of the number of Employee objects that have been created.



Software Engineering Observation 10.12

*Actually, `string` objects in C# are **immutable**—they cannot be modified after they're created. Therefore, it's safe to have many references to one `string`. This is not normally the case for objects of most other classes. If `string` objects are immutable, you might wonder why we're able to use operators `+` and `+=` to concatenate strings. String-concatenation operations actually result in a new `string` object containing the concatenated values. The original string objects are not modified.*

```
1  // Fig. 10.11: EmployeeTest.cs
2  // static member demonstration.
3  using System;
4
5  class EmployeeTest
6  {
7      static void Main()
8      {
9          // show that Count is 0 before creating Employees
10         Console.WriteLine(
11             $"Employees before instantiation: {Employee.Count}");
12
13         // create two Employees; Count should become 2
14         var e1 = new Employee("Susan", "Baker");
15         var e2 = new Employee("Bob", "Blue");
16
17         // show that Count is 2 after creating two Employees
18         Console.WriteLine(
19             $"\\nEmployees after instantiation: {Employee.Count}");
```

Fig. 10.11 | static member demonstration. (Part 1 of 2.)


```
20
21 // get names of Employees
22 Console.WriteLine($"Employee 1: {e1.FirstName} {e1.LastName}");
23 Console.WriteLine($"Employee 2: {e2.FirstName} {e2.LastName}");
24
25 // in this example, there is only one reference to each Employee,
26 // so the following statements cause the CLR to mark each
27 // Employee object as being eligible for garbage collection
28 e1 = null; // mark object referenced by e1 as no longer needed
29 e2 = null; // mark object referenced by e2 as no longer needed
30 }
31 }
```

Employees before instantiation: 0
Employee constructor: Susan Baker; Count = 1
Employee constructor: Bob Blue; Count = 2

Employees after instantiation: 2

Employee 1: Susan Baker
Employee 2: Bob Blue

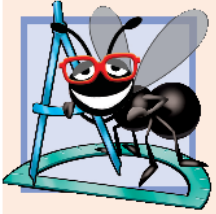
Fig. 10.11 | static member demonstration. (Part 2 of 2.)

10.10 readonly Instance Variables

- ▶ The principle of least privilege states that code should be granted the amount of privilege and access needed to accomplish its designated task, but no more.
- ▶ Constants declared with `const` must be initialized to a constant value when they're declared.
- ▶ C# provides keyword `readonly` to specify that an instance variable of an object is not modifiable and that any attempt to modify it after the object is constructed is an error.
- ▶ A readonly variable's identifier uses Pascal Case by convention.

10.10 readonly Instance Variables (Cont.)

- ▶ Although `readonly` instance variables can be initialized when they're declared, this isn't required.
- ▶ A `readonly` variable should be initialized *by each* of the class's constructors or in the variable's declaration.
- ▶ Each constructor can assign values to a `readonly` instance variable multiple times—the variable doesn't become unmodifiable until *after* the constructor completes execution.
- ▶ If a constructor does not initialize the `readonly` variable, the variable uses the same default value as any other instance variable.



Software Engineering Observation 10.13

Declaring an instance variable as `readonly` helps enforce the principle of least privilege. If an instance variable should not be modified after the object is constructed, declare it to be `readonly` to prevent modification.

10.10 readonly Instance Variables (Cont.)

- ▶ `const` members must be assigned values at compile time.
- ▶ `const` members can be initialized *only* with other constant values, such as integers, `string` literals, characters and other `const` members.
- ▶ Constant members with values that cannot be determined at compile time—such as constants that are initialized with the result of a method call—must be declared with keyword `readonly`, so they can be initialized at *execution time*.
- ▶ Variables that are `readonly` can be initialized with more complex expressions, such as an array initializer or a method call that returns a value or a reference to an object.



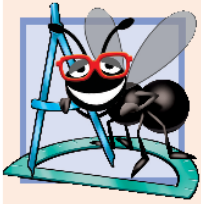
Common Programming Error 10.3

Attempting to modify a readonly instance variable anywhere but in its declaration or the object's constructors is a compilation error.



Error-Prevention Tip 10.1

Attempts to modify a readonly instance variable are caught at compilation time rather than causing execution-time errors. It's always preferable to get bugs out at compile time, if possible, rather than allowing them to slip through to execution time (where studies have found that repairing bugs is often much more costly).



Software Engineering Observation 10.14

If a `readonly` instance variable is initialized to a constant only in its declaration, it's not necessary to have a separate copy of the instance variable for every object of the class. The variable should be declared `const` instead. Constants declared with `const` are implicitly `static`, so there will only be one copy for the entire class.

10.10 readonly Instance Variables (Cont.)

C# 6 Getter-Only Auto-Implemented Properties and readonly

- ▶ When an auto-implemented property has only a `get` accessor, the property can be used only to read the value, so the compiler implicitly declares the corresponding `private` instance variable as `readonly`.
- ▶ Getter-only auto-implemented properties can be initialized in their declarations or in constructors.

10.11.1 Using the Class View Window

- The **Class View** displays the fields, methods and properties for all classes in a project.
- Select **View > Class View** to display the **Class View** as a tab in the same position within the IDE
- Figure 10.12 shows the **Class View** for the `Time1` project of Fig. 10.1 (class `Time1`) and Fig. 10.2 (class `TimeTest1`).

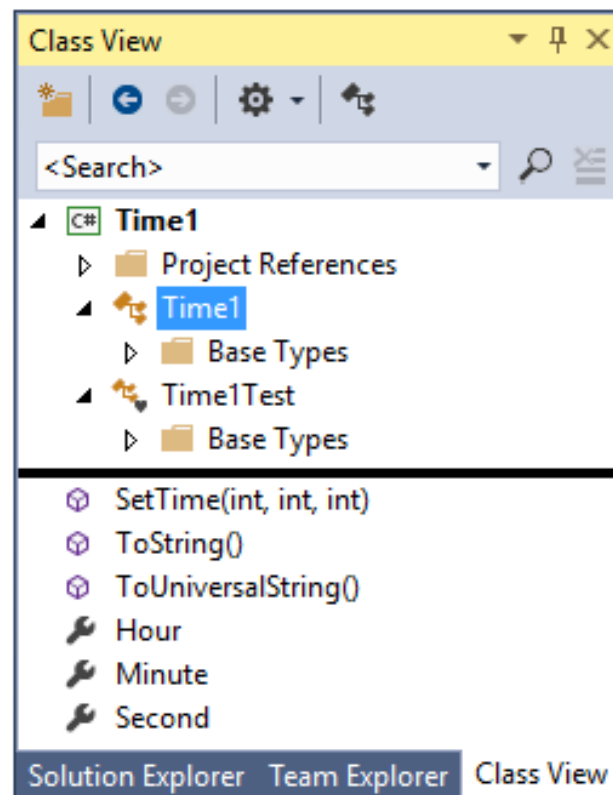


Fig. 10.12 | **Class View** of class **Time1** (Fig. 10.1) and class **Time1Test** (Fig. 10.2).

10.11.1 Using the Class View Window

- The view follows a hierarchical structure, with the project name as the root.
- When a class is selected, its members appear in the lower half of the window.
- ▶ `public` methods are indicated by purple boxes
- ▶ `public` properties are indicated by wrench icons
- ▶ If a class has any `private` members, those members' icons contain small padlocks.

10.11.2 Using the Object Browser

- You can use the **Object Browser** to learn about the functionality provided by a specific class.
- To open the **Object Browser**, select **View > Object Browser**.
- Figure 10.13 depicts the **Object Browser** when the user navigates to the `Math` class in namespace `System` in the assembly `mscorlib.dll` (Microsoft Core Library).

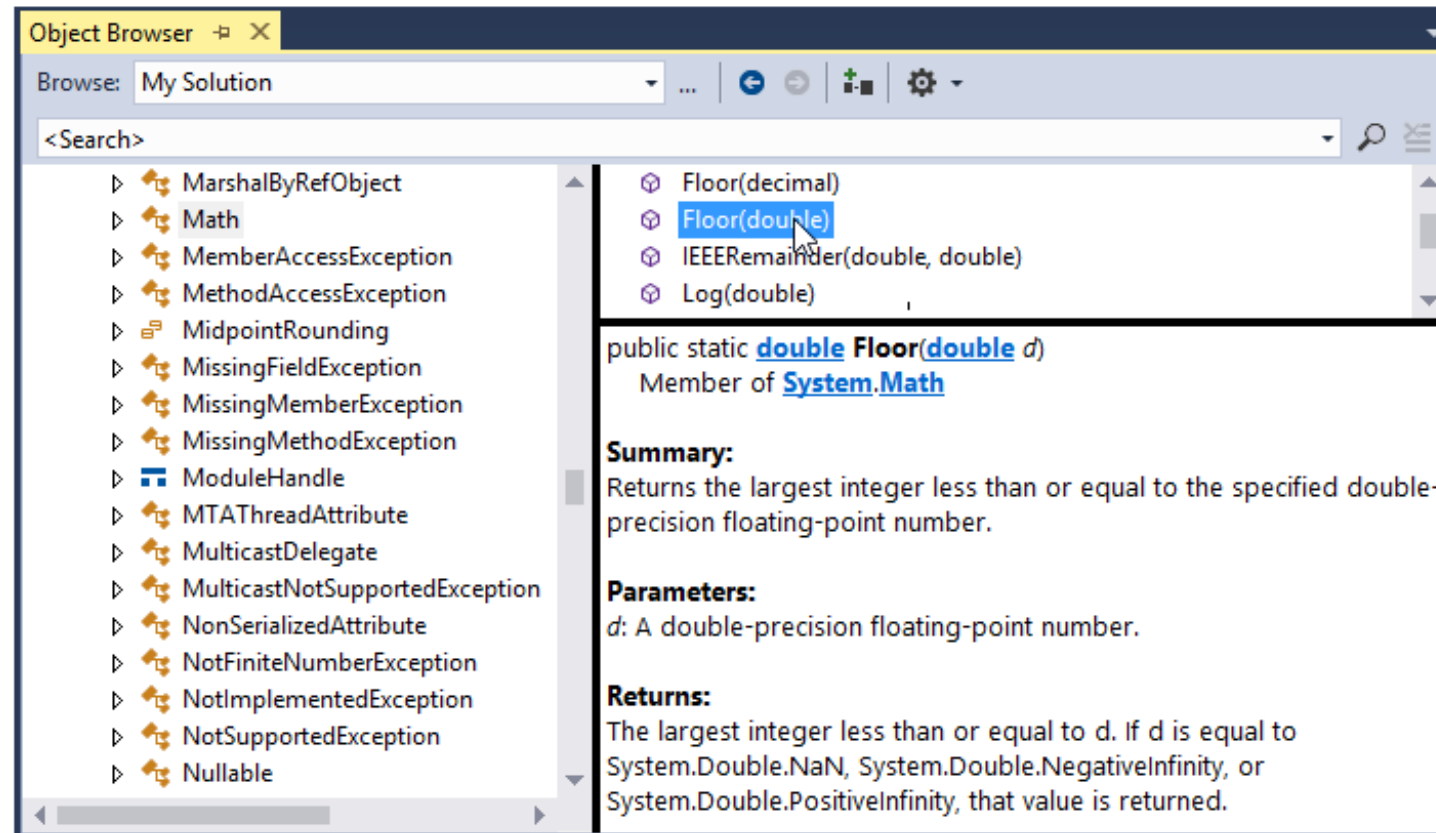


Fig. 10.13 | Object Browser for class Math.

10.11.2 Using the Object Browser

- The **Object Browser** lists all methods provided by class `Math` in the upper-right frame.
- If you click the name of a member in the upper-right frame, a description of that member appears in the lower-right frame.
- The **Object Browser** can be a quick mechanism to learn about a class or one of its methods.

10.12 Object Initializers

- Object initializers allow you to create an object and initialize its `public` properties (and `public` instance variables, if any) in the same statement.
- This can be useful when a class does not provide an appropriate constructor to meet your needs, but does provide a constructor that can be called with no arguments and properties that you can use to set the class's data
- `new Time2` is immediately followed by an object-initializer list—a comma-separated list in curly braces (`{ }`) of properties and their values.

10.12 Object Initializers (Cont.)

- Each property name can appear only once in the object-initializer list.
- The object initializer executes the property initializers in the order in which they appear.

10.13 Operator Overloading; Introducing struct

- ▶ You can create operators that work with objects of your own types—via a process called **operator overloading**.
- ▶ You can overload most operators.

10.13.1 Creating Value Types with struct

- ▶ C#'s simple numeric types are value types.
- ▶ C#'s simple types like `int` and `double` are actually aliases for `struct` types—an `int` is defined by the `struct System.Int32`, a `long` by `System.Int64`, a `double` by `System.Double` and so on.
- ▶ Operator overloading also can be applied to classes.

10.13.1 Creating Value Types with struct

When to Declare a struct Type

- ▶ Microsoft recommends using classes for most new types, but recommends a `struct` if:
 - the type represents a single value
 - the size of an object is 16 bytes or smaller
- ▶ For the complete list of struct recommendations, see
 - <https://msdn.microsoft.com/library/ms229017>

10.13.2 Creating Value Types with struct

- ▶ Value type `ComplexNumber` (Fig. 10.14) overloads the plus (+), minus (-) and multiplication (*) operators to enable programs to add, subtract and multiply instances of class `ComplexNumber` using common mathematical notation.

```
1  // Fig. 10.14: ComplexNumber.cs
2  // Value type that overloads operators for adding, subtracting
3  // and multiplying complex numbers.
4  using System;
5
6  public struct ComplexNumber
7  {
8      // read-only properties that get the real and imaginary components
9      public double Real { get; }
10     public double Imaginary { get; }
11
12     // constructor
13     public ComplexNumber(double real, double imaginary)
14     {
15         Real = real;
16         Imaginary = imaginary;
17     }
```

Fig. 10.14 | Value type that overloads operators for adding, subtracting and multiplying complex numbers. (Part 1 of 3.)

```
18
19 // return string representation of ComplexNumber
20 public override string ToString() =>
21     $"({Real} {(Imaginary < 0 ? "-" : "+"}) {Math.Abs(Imaginary)}i)";
22
23 // overload the addition operator
24 public static ComplexNumber operator+(ComplexNumber x, ComplexNumber y)
25 {
26     return new ComplexNumber(x.Real + y.Real,
27         x.Imaginary + y.Imaginary);
28 }
29
```

Fig. 10.14 | Value type that overloads operators for adding, subtracting and multiplying complex numbers. (Part 2 of 3.)

```
30 // overload the subtraction operator
31 public static ComplexNumber operator-(ComplexNumber x, ComplexNumber y)
32 {
33     return new ComplexNumber(x.Real - y.Real,
34         x.Imaginary - y.Imaginary );
35 }
36
37 // overload the multiplication operator
38 public static ComplexNumber operator*(ComplexNumber x, ComplexNumber y)
39 {
40     return new ComplexNumber(
41         x.Real * y.Real - x.Imaginary * y.Imaginary,
42         x.Real * y.Imaginary + y.Real * x.Imaginary);
43 }
44 }
```

Fig. 10.14 | Value type that overloads operators for adding, subtracting and multiplying complex numbers. (Part 3 of 3.)

10.13.2 Creating Value Types with struct

- ▶ Unlike a class, you cannot define a parameterless constructor for a `struct`.
- ▶ The compiler always provides a default constructor that initializes the `struct`'s instance variables to their default values.
- ▶ `structs` cannot specify initial values in instance variable or property declarations.

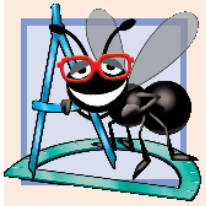
10.13.2 Creating Value Types with struct

- ▶ Keyword **operator**, followed by an operator symbol (such as +), indicates that a method overloads the specified operator.
- ▶ Overloaded operator methods are *required* to be `public` and `static`.
- ▶ Methods that overload binary operators must take two arguments—the *first* is the *left* operand and the *second* is the *right* operand.



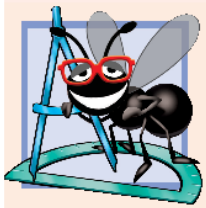
Software Engineering Observation 10.15

Overload operators to perform the same function or similar functions on objects as the operators perform on objects of simple types. Avoid nonintuitive use of operators.



Software Engineering Observation 10.16

At least one parameter of an overloaded operator method must be of the type in which the operator is overloaded. This prevents you from changing how operators work on simple types.



Software Engineering Observation 10.17

Though you cannot overload the arithmetic assignment operators (e.g., `+=` and `-=`), C# allows you to use them with any type that declares the corresponding arithmetic operator (e.g., `+` and `-`).

10.13.3 Class ComplexTest

- ▶ Class `ComplexTest` (Fig. 10/15) demonstrates the overloaded operators for adding, subtracting and multiplying `ComplexNumbers`.

```
1  // Fig. 10.15: ComplexTest.cs
2  // Overloading operators for complex numbers.
3  using System;
4
5  class ComplexTest
6  {
7      static void Main()
8      {
9          // prompt the user to enter the first complex number
10         Console.Write("Enter the real part of complex number x: ");
11         double realPart = double.Parse(Console.ReadLine());
12         Console.Write("Enter the imaginary part of complex number x: ");
13         double imaginaryPart = double.Parse(Console.ReadLine());
14         var x = new ComplexNumber(realPart, imaginaryPart);
15
16         // prompt the user to enter the second complex number
17         Console.Write("\nEnter the real part of complex number y: ");
18         realPart = double.Parse(Console.ReadLine());
19         Console.Write("Enter the imaginary part of complex number y: ");
20         imaginaryPart = double.Parse(Console.ReadLine());
21         var y = new ComplexNumber(realPart, imaginaryPart);
22
23         // display the results of calculations with x and y
24         Console.WriteLine();
```

Fig. 10.15 | Overloading operators for complex numbers. (Part I of 2.)

```
25 Console.WriteLine($"{x} + {y} = {x + y}");
26 Console.WriteLine($"{x} - {y} = {x - y}");
27 Console.WriteLine($"{x} * {y} = {x * y}");
28     }
29 }
```

```
Enter the real part of complex number x: 2
Enter the imaginary part of complex number x: 4

Enter the real part of complex number y: 4
Enter the imaginary part of complex number y: -2

(2 + 4i) + (4 - 2i) = (6 + 2i)
(2 + 4i) - (4 - 2i) = (-2 + 6i)
(2 + 4i) * (4 - 2i) = (16 + 12i)
```

Fig. 10.15 | Overloading operators for complex numbers. (Part 2 of 2.)

10.14 Time Class Case Study: Extension Methods

- ▶ You can use **extension methods** to add functionality to an existing type without modifying the type's source code.
- ▶ LINQ's capabilities are implemented as extension methods.
- ▶ Figure 10.16 uses extension methods to add two new methods to class `Time2` (Section 10.5)—`DisplayTime` and `AddHours`.

```
1  // Fig. 10.16: TimeExtensionsTest.cs
2  // Demonstrating extension methods.
3  using System;
4
5  class TimeExtensionsTest
6  {
7      static void Main()
8      {
9          var myTime = new Time2(); // call Time2 constructor
10         myTime.SetTime(11, 34, 15); // set the time to 11:34:15
11
12         // test the DisplayTime extension method
13         Console.WriteLine("Use the DisplayTime extension method: ");
14         myTime.DisplayTime();
15
16         // test the AddHours extension method
17         Console.WriteLine("Add 5 hours with the AddHours extension method: ");
18         var timeAdded = myTime.AddHours(5); // add five hours
19         timeAdded.DisplayTime(); // display the new Time2 object
```

Fig. 10.16 | Demonstrating extension methods. (Part 1 of 4.)

```
20
21 // add hours and display the time in one statement
22 Console.WriteLine("Add 15 hours with the AddHours extension method: ");
23 myTime.AddHours(15).DisplayTime(); // add hours and display time
24
25 // use fully qualified extension-method name to display the time
26 Console.WriteLine("Use fully qualified extension-method name: ");
27 TimeExtensions.DisplayTime(myTime);
28     }
29 }
30
```

Fig. 10.16 | Demonstrating extension methods. (Part 2 of 4.)

```
31 // extension-methods class
32 static class TimeExtensions
33 {
34     // display the Time2 object in console
35     public static void DisplayTime(this Time2 aTime)
36     {
37         Console.WriteLine(aTime.ToString());
38     }
39
40     // add the specified number of hours to the time
41     // and return a new Time2 object
42     public static Time2 AddHours(this Time2 aTime, int hours)
43     {
44         // create a new Time2 object
45         var newTime = new Time2() {
46             Minute = aTime.Minute, Second = aTime.Second};
47
48         // add the specified number of hours to the given time
49         newTime.Hour = (aTime.Hour + hours) % 24;
50
51         return newTime; // return the new Time2 object
52     }
53 }
```

Fig. 10.16 | Demonstrating extension methods. (Part 3 of 4.)

```
Use the DisplayTime extension method: 11:34:15 AM
Add 5 hours with the AddHours extension method: 4:34:15 PM
Add 15 hours with the AddHours extension method: 2:34:15 AM
Use fully qualified extension-method name: 11:34:15 AM
```

Fig. 10.16 | Demonstrating extension methods. (Part 4 of 4.)

10.14 Time Class Case Study: Extension Methods (Cont.)

- ▶ The `this` keyword before a method's first parameter notifies the compiler that the method extends an existing class.
- ▶ The type of the first parameter specifies the type of object on which you can call the extension method.
- ▶ Extension methods must be defined as `static` methods in a `static` class.
- ▶ A `static` class can contain only `static` members and cannot be instantiated.

10.14 Time Class Case Study: Extension Methods (Cont.)

- ▶ When you call an extension method, you do not provide an argument for the first parameter.
- ▶ The compiler implicitly passes the object that calls the method as the extension method's first argument.
- ▶ This allows you to call the extension method as if it were an instance method.
- ▶ *IntelliSense* displays extension methods with the class's instance methods and identifies them with a similar icon that also has a down arrow.

10.14 Time Class Case Study: Extension Methods (Cont.)

- ▶ Line 23 uses both extension methods (`DisplayTime` and `AddHours`) in a single statement to add 15 hours to the original `myTime` and display the result in the console.
- ▶ Multiple method calls in the same statement are known as **cascaded method calls**.
- ▶ When a method returns an object, you can follow the method call with a member access operator (`.`) then call a method on the object that was returned.
- ▶ The methods are called from left to right.

10.14 Time Class Case Study: Extension Methods (Cont.)

- ▶ You can call an extension method via its fully qualified name by using the name of the class in which the extension method is defined (TimeExtensions), followed by the member access operator (.), the method name (DisplayTime) and its argument list.
- ▶ When using the fully qualified method name, you must specify an argument for extension method's first parameter.

10.14 Time Class Case Study: Extension Methods (Cont.)

- ▶ If a type for which you declare an extension method already defines an instance method with the same name and a compatible signature, the instance method will shadow (i.e., hide) the extension method.
- ▶ If a predefined type is later updated to include an instance method that shadows an extension method, the compiler does not report any errors and the extension method does not appear in *IntelliSense*.